PrivacyGuard Detector 地址检测方法论与英文地址审核

1. 文档范围

本文基于当前仓库中的**真实实现**整理地址检测方法，而不是复述旧设计草案。核心依据如下：

- `privacyguard/infrastructure/pii/detector/scanner.py`
- `privacyguard/infrastructure/pii/detector/stacks/address.py`
- `privacyguard/utils/normalized_pii.py`
- `tests/test_address_stack.py`
- `tests/test_address_stack_refactor.py`
- `tests/test_normalized_pii_address_en.py`

本文分两部分：

1. 以论文“方法论”章节的写法，描述当前**中文地址检测**算法。
2. 审核当前**英文地址检测/归一**方案，分析其与中文地址结构的差异，并判断哪些机制可以复用，哪些必须正向改造。

---

2. 中文地址检测方法

2.1 问题定义

给定一段文本流 $X = \{u_1, \ldots, u_n\}$，其中每个 `unit` 由预处理器按中文单字、英文单词、数字串、空白和标点切分得到，系统首先从扫描器中获得地址相关 clue 序列：

- `VALUE`：行政区划值，如省、市、区。
- `KEY`：地址关键词，如“路”“街”“号”“栋”“室”。
- `LABEL`：如“收货地址”“地址”等字段标签。
- `NEGATIVE/BREAK`：负向语义和硬断点。

中文地址检测的目标不是直接做整串正则匹配，而是在 clue 序列上恢复一个**结构化组件链**：

$$A = [c_1, c_2, \ldots, c_m], \quad c_i \in \{\text{province}, \text{city}, \text{district}, \text{subdistrict}, \text{road}, \text{number}, \text{poi}, \text{building}, \text{detail}\}$$

并最终输出：

- 地址文本范围；
- 组件序列；
- 组件 trace；
- 组件 key trace；
- suspect/suspected 元数据。

上游 clue 生成方式如下：

- 中文行政值由 `scanner.py` 中 `_zh_address_value_matcher()` 从 `zh_geo_lexicon.json` 生成。
- 中文地址关键词由 `_zh_address_key_matcher()` 从 `zh_address_keywords.json` 生成。

对应实现见：

- `privacyguard/infrastructure/pii/detector/scanner.py:845-887`
- `privacyguard/infrastructure/pii/detector/scanner.py:1441-1474`

2.2 状态表示

地址解析过程由 `AddressStack` 中的 `_ParseState` 驱动。关键状态包括：

- `components`：已经提交的地址组件。
- `deferred_chain`：尚未落地、等待后续 clue 消费的链。
- `suspect_chain`：链上可进入 `suspected` 的行政 `VALUE`。
- `occupancy`：单占位组件是否已出现。
- `segment_state`：当前段的方向状态，特别用于逗号尾。
- `last_consumed / last_end`：最近消费 clue 与其右边界。
- `absorbed_digit_unit_end`：已跨过的数字 clue 终点。

其中，以下类型在同一地址实例内只能出现一次：

- `province`
- `city`
- `district`
- `subdistrict`
- `road`
- `number`

对应实现见：

- `privacyguard/infrastructure/pii/detector/stacks/address.py:50-80`
- `privacyguard/infrastructure/pii/detector/stacks/address.py:198-234`

2.3 起栈与种子选择

解析入口 `AddressStack.run()` 支持两类起点：

1. `LABEL/START` 驱动起栈。
2. `VALUE/KEY` 直接起栈。

当由标签起栈时，系统会在标签右侧跳过分隔符，并在 **6 个 unit** 范围内寻找最近的地址 seed。若找不到，则不生成地址候选。该机制避免了“字段标签存在，但值域为空”时的误报。

对应实现见：

- `privacyguard/infrastructure/pii/detector/stacks/address.py:1380-1418`
- `privacyguard/infrastructure/pii/detector/stacks/address.py:1938-1950`

2.4 主循环：基于 clue 状态机的地址恢复

主循环 `_run_with_clues()` 在 clue 序列上从左向右推进，并维持一个**局部连续性约束**：

$$\mathrm{gap}(clue_i, clue_{i+1}) \leq 6 \text{ units}$$

一旦超出该局部窗口，当前 run 停止。该约束用于抑制远距离误拼接，也允许 OCR/插入数字造成的小范围扰动。

对应实现见：

- `privacyguard/infrastructure/pii/detector/stacks/address.py:1505-1641`
- `privacyguard/infrastructure/pii/detector/stacks/address.py:247-257`

2.4.1 链式缓冲 `deferred_chain`

算法并不在看到每个 clue 时立即提交组件，而是优先把 clue 放入 `deferred_chain`，等到链闭合或断开时统一决定：

- 若链尾存在 `KEY`，则由**最后一个 KEY** 决定最终组件类型。
- 若链中只有 `VALUE`，则这些 `VALUE` 逐个独立落地。

这种设计使系统能够表达“VALUE→VALUE→KEY”“KEY→KEY 穿透”等非局部组合。

对应实现见：

- `privacyguard/infrastructure/pii/detector/stacks/address.py:892-957`
- `privacyguard/infrastructure/pii/detector/stacks/address.py:960-985`

2.4.2 链接受规则

`_chain_can_accept()` 定义了链上传导的四类规则：

1. `VALUE -> VALUE`：间隔不超过 1 个非空白 unit。
2. `VALUE -> KEY`：间隔不超过 6 个 unit。
3. `KEY -> KEY`：允许紧邻，或 gap=1 且满足正文穿透规则。
4. `KEY -> VALUE`：禁止。

这意味着中文地址可以自然处理如下结构：

- “上海 + 路”
- “科技园 + 社区 + 小区”
- “北京市 + 朝阳区 + 建国路 + 88号”

对应实现见：

- `privacyguard/infrastructure/pii/detector/stacks/address.py:463-483`
- `privacyguard/infrastructure/pii/detector/stacks/address.py:292-330`

2.4.3 KEY 的动态路由

部分中文关键词具有上下文歧义，系统通过 `_route_dynamic_key_type()` 做动态路由，例如：

- “社区”在某些上下文中转成 `poi`。
- “楼”在纯字母数字左值下，可能被解释为 `building` 或 `detail`，而不是 `poi`。

这一步并不是简单词典分类，而是“词典类型 + 上下文 + 后继 clue”的联合判定。

对应实现见：

- `privacyguard/infrastructure/pii/detector/stacks/address.py:653-679`
- `privacyguard/infrastructure/pii/detector/stacks/address.py:824-889`

2.5 组件提交：占位约束与后继图

`_commit()` 在组件提交时执行两层约束：

1. **单占位冲突检查**：同层级重复出现则切分地址边界。
2. **后继图合法性检查**：当前组件必须满足层级后继关系。

对于普通段，系统要求组件按后继图前向推进；对于逗号尾段，允许在首个新组件后锁定方向，并在该方向内继续扩展。

对应实现见：

- `privacyguard/infrastructure/pii/detector/stacks/address.py:1016-1050`
- `privacyguard/infrastructure/pii/detector/stacks/address.py:1085-1115`
- `privacyguard/infrastructure/pii/detector/stacks/address.py:83-101`

2.6 逗号尾机制

中文地址的一个重要现象是：主地址之后常出现“逗号 + 更高层级行政链”，例如：

- `金钟路968号,上海市`

对此，算法引入了逗号尾预处理 `_comma_tail_prehandle()`：

1. 先在 gap 内寻找逗号。
2. 如有链未落地，先 flush 左链。
3. 预演逗号后首个真实组件类型。
4. 只有当该首组件属于 `district/city/province` 且层级高于逗号左侧最高行政层时，才允许进入逗号尾模式。
5. 若逗号尾最终落到区以下层级，则回滚到最近的逗号左侧快照。

这使得算法能够接受“逆序补充更高层级”，但拒绝“逗号后掉回 road/poi/building/detail”的非法延伸。

对应实现见：

- `privacyguard/infrastructure/pii/detector/stacks/address.py:2129-2187`
- `privacyguard/infrastructure/pii/detector/stacks/address.py:2014-2126`
- `privacyguard/infrastructure/pii/detector/stacks/address.py:416-438`

2.7 负向 clue 裁剪

若地址右端或中间与负向 clue 重叠，算法不会直接整条地址作废，而是从右向左弹出与负向 span 重叠的组件，直到最右组件不再与负向区域相交为止。

这种右弹策略的核心效果是：

- 允许中间存在噪声但保住有效尾部。
- 若最右端组件被负向词覆盖，则逐步缩短地址。

对应实现见：

- `privacyguard/infrastructure/pii/detector/stacks/address.py:2383-2397`

2.8 `suspected` 后处理

对于通过链式吸收形成的组件，系统会把其左侧行政 `VALUE` clue 提取为 `suspected` 元数据，并从主值中删除这些行政文字。于是一个组件可以同时承载：

- `value`：真正的核心地址值。
- `suspected`：前置行政层信息。

例如，道路组件可以携带：

- `value = 中山`
- `suspected = {city: 北京, district: 朝阳}`

这样做的意义在于：检测阶段保持完整上下文，比对阶段又能把组件主值与行政补充信息分开处理。

对应实现见：

- `privacyguard/infrastructure/pii/detector/stacks/address.py:1195-1239`
- `privacyguard/infrastructure/pii/detector/stacks/address.py:2332-2376`

2.9 数字尾部补全 `digit_tail`

中文地址中，“路79”“A栋1203”“10-2-301”等数字尾通常不是独立 clue 链，而是紧贴在上一组件后。为此，系统在主循环收尾阶段执行 `_analyze_digit_tail()`：

1. 读取最后一个组件后的下一个 `digit_run`。
2. 按前一组件类型决定允许的尾部层级和连字符数量。
3. 贪心把数字尾切分并分配到 `building/detail` 等层级。
4. 若数字尾后方紧接地址 KEY，则暂不物化，等待更完整的地址结构。

该步骤增强了“楼栋/房间号”一类细粒度组件的召回。

对应实现见：

- `privacyguard/infrastructure/pii/detector/stacks/address.py:2549-2628`
- `privacyguard/infrastructure/pii/detector/stacks/address.py:2529-2546`

2.10 HARD clue 子分词

当地址来源是高置信 `HARD` clue 时，系统不会直接把整段文本当作最终地址，而是调用 `_sub_tokenize()` 在该 span 内二次扫描地址 value/key 词典，并把生成的 sub-clue 重新送入同一套状态机。

因此，HARD 路径并不是另一套解析器，而是“局部 clue 重建 + 同一主循环”。

对应实现见：

- `privacyguard/infrastructure/pii/detector/stacks/address.py:1246-1339`
- `privacyguard/infrastructure/pii/detector/stacks/address.py:1420-1502`

2.11 输出形式

最终输出候选包含：

- `address_component_type`
- `address_component_trace`
- `address_component_key_trace`
- `address_details_type`
- `address_details_text`
- `address_component_suspected`

这些元数据是后续 `normalize_pii(ADDRESS)` 和 `same_entity()` 的直接输入。

对应实现见：

- `privacyguard/infrastructure/pii/detector/stacks/address.py:1885-1931`
- `privacyguard/infrastructure/pii/detector/stacks/address.py:2332-2376`

---

3. 英文地址实现审核

3.1 当前英文方案的真实结构

当前英文地址不是“另一套独立 parser”，而是由三层共同组成：

1. **Scanner 层**：
    - `VALUE` 仅从英文地理词典中产出 `province/state` 与 `city`。
    - `KEY` 仅从 `en_address_keywords.json` 产出 `road/detail/building/poi`。
    - 邮编还额外用 `_POSTAL_CODE_PATTERN` 扫描为一个英文地址 `VALUE(detail)`。
2. **AddressStack 层**：
    - 仍然复用与中文相同的 `AddressStack`。
    - 只在少数细节上按 locale 分支，如英文左扩、前缀型关键词判断。
3. **归一化层**：
    - `normalize_pii(ADDRESS)` 只接受 detector metadata 或显式 `components`。
    - 不再从 `raw_text` 做内部正则兜底。

对应实现见：

- `privacyguard/infrastructure/pii/detector/scanner.py:890-948`
- `privacyguard/infrastructure/pii/detector/scanner.py:1478-1509`
- `privacyguard/infrastructure/pii/detector/stacks/address.py:1380-1810`
- `privacyguard/utils/normalized_pii.py:187-245`
- `privacyguard/utils/normalized_pii.py:760-774`

需要特别指出的是，仓库里仍保留了一套旧的英文地址解析辅助逻辑，例如：

- `privacyguard/utils/pii_value.py:53-60`
- `privacyguard/utils/pii_value.py:726-795`

其中已经接入了：

- 州名/州缩写别名；
- 国家别名；
- 英文街道与单元前缀模式。

但这套能力**不在当前 `normalize_pii(ADDRESS)` 的有效路径上**。也就是说，英文地址的别名映射和正则结构化知识，目前大部分是“仓库里存在”，但“当前主链路没有真正使用”。

3.2 英文地址现状的实测结论

我基于当前代码实际跑了几条英文地址，得到如下结果。

样例 1

输入：

```
Apt 205, 5622 Lincoln Avenue, Seattle, WA 45283
```

当前输出被拆成两个地址候选：

1. `Apt 205`
2. `Lincoln Avenue, Seattle, WA 45283`

其中第二条候选的组件 trace 为：

```
road:Lincoln
city:Seattle
province:WA 45283
```

问题：

- `Apt 205` 被错误拆成独立地址。
- 门牌号 `5622` 丢失。
- 邮编 `45283` 被并入 `province`，没有单独建模。

样例 2

输入：

```
1425 Pine St Apt 301, Seattle, WA 98122
```

当前输出：

```
Pine St Apt 301, Seattle, WA 98122
```

组件 trace：

```
detail:PineSt
city:Seattle
province:WA 98122
```

问题：

- 门牌号 `1425` 丢失。
- `road` 没有被建模。
- `St -> Apt` 的 KEY→KEY 传导把街道错误吞进了 `detail`。
- 邮编再次被吸进 `province`。

样例 3

输入：

```
620 8th Ave, New York, NY 10018
```

当前输出：

```
New York, NY 10018
```

问题：

- `620 8th Ave` 整段街道部分被完全漏掉。
- 原因不是 city/state 没识别，而是 street number 与 ordinal road core 在上游/左扩阶段丢失了。

样例 4

输入：

```
2900 Sunset Blvd Apt 4B, Los Angeles, CA 90026
```

当前输出：

```
Sunset Blvd Apt 4B, Los Angeles, CA 90026
```

组件 trace：

```
detail:SunsetBlvd
city:Los Angeles
province:CA 90026
```

问题：

- 门牌号 `2900` 丢失。
- `road` 再次被 detail 吞并。
- `Apt` 没有作为“右结合的单元前缀”处理，而是把前面 street chain 当成了 detail 的 value。

3.3 英文测试与实现之间的脱节

当前仓库中，英文地址的自动化测试主要集中在**归一化**，而不是 detector 本身：

- `tests/test_normalized_pii_address_en.py`

但这些测试中还存在一个当前即失败的断言。实际运行：

```
C:\Users\vis\.conda\envs\paddle\python.exe -m pytest tests\test_normalized_pii_address_en.py
-q
```

结果是：

- `3` 个测试里 `1` 个失败。
- 失败点为 `same_entity(left, right)` 没有把 `California` 与 `CA` 判为同一州。

这说明“英文州名/缩写等价”目前**并未打通到生效路径**。

同时，中文地址 stack 的测试已经覆盖大量链式、逗号尾、digit tail、suspected 情况，而英文 detector 几乎没有对应的 stack 级回归测试。这意味着：

- 英文归一化有少量测试。
- 英文检测主链路几乎没有被系统性约束。

除此之外，现有评估文件 `outputs/analysis/andlab_persona_eval_address_cases.csv` 中，按 `locale=us` 读取可得到：

- 总样本数：`240`
- `any_address=True`：`236`
- `partial_hit=True`：`233`
- `exact_full_address=True`：`0`

这说明当前英文链路更像是**地址片段召回器**，而不是**完整英文地址结构恢复器**。也就是说，系统经常能抓到“与地址相关的片段”，但几乎不能稳定还原整条英文地址。

3.4 英文地址与中文地址的结构差异

中文与英文地址在结构上有根本差异。

中文地址的典型性质

- 常按**行政层级从大到小**书写。
- 关键词通常位于值的**右侧后缀**，如“中山路”“朝阳区”“3号”“2栋”“301室”。
- 无逗号连续书写很常见。
- 单个 KEY 可以天然承担“层级结束标志”的作用。

英文地址的典型性质

- 常见主链是：`house number -> street core -> street suffix`。
- 单元信息大量使用**前缀型关键词**，如 `Apt 301`、`Suite 300`、`Unit 18`、`Floor 8`。
- `city, state ZIP` 常由逗号和空格分层，不是中文那种连续 suffix 链。
- 州名既可能写全称，也可能写缩写。
- 街道还包含方向词、序数词、缩写、PO Box、复合专名等。

因此，英文地址不是“把中文词典换成英文词典”就能成立；其组合语法本身就不同。

---

4. 是否能复用中文检测方法

4.1 结论

可以复用框架，不能直接复用规则。

更准确地说：

- **可复用**的是：`clue -> state machine -> components -> metadata -> normalize_pii` 这一整体架构。
- **不可直接复用**的是：中文那套“右侧 suffix key 驱动的链式吞噬规则”。

中文方法的核心假设是：

1. 关键词主要在右侧。
2. VALUE→KEY 或 KEY→KEY 穿透通常能让结构更完整。
3. 行政层与道路层的合法后继图较稳定。

而英文地址至少有三条假设不成立：

1. `Apt/Suite/Unit/Floor/Room` 是**前缀 key**。
2. `road suffix` 与 `detail prefix` 连续出现时，不能简单按中文 KEY→KEY 继续穿透。
3. `house number / ZIP` 是结构上非常重要的成分，但当前主链路里它们要么被 structured clue 抢走，要么被吸进别的组件。

因此，中文方法只能作为**骨架复用**，不能作为**语法规则直接平移**。

4.2 可以直接复用的部分

以下机制仍然值得保留：

- `unit` 切分与 6-unit 局部连续性约束。

- `LABEL` 驱动起栈。
- `BREAK / NEGATIVE` 的中断机制。
- `deferred_chain` 的链式缓冲思想。
- `occupancy` 与组件 metadata 输出。
- parser 层的冲突裁决、shrink 与相邻地址吸收框架。

4.3 不能直接复用的部分

以下机制需要英语专用改写：

1. `KEY -> KEY` 穿透规则。
2. 英文左扩的 floor 规则。
3. 逗号尾 `VALUE` 的右向扫描。
4. 把邮编当成普通 numeric/ALNUM 吸收的策略。
5. 把州名/州缩写比较建立在简单子串包含上的策略。

---

5. 英文地址应如何正向改动

下面给出我认为正确的正向改造方向。重点是**把英文当成一套独立语法**，而不是继续给中文规则打补丁。

5.1 在 scanner 层补齐英文结构信号

当前英文 scanner 只稳定产出：

- `city`
- `province/state`
- `road/detail/building/poi` 关键词

但英文完整地址至少还需要显式建模：

- `house_number`
- `postal_code`
- `country`
- 可选 `directional`（N/S/E/W/NE/NW/SE/SW）
- 可选 `street_core`

建议：

1. 保留现有 `road/detail/building/poi` 关键词扫描。
2. 新增 `house_number` 与 `postal_code` 的地址 clue，不要让它们只作为 structured numeric 存在。
3. 让邮编 clue 在地址场景中优先保留，而不是被 numeric 硬 clue 抢走。

5.2 把英文 KEY 分成“后缀型”和“前缀型”

这是最关键的结构改造。

建议把英文地址 key 至少分成两类：

- **Suffix key**：`Street/St/Road/Rd/Avenue/Ave/Blvd/...`
- **Prefix key**：`Apt/Suite/Ste/Unit/Floor/Fl/Room/Rm/#`

两类 key 应采用不同结合方向：

- `suffix key` 向左结合，例如 `8th Ave`、`Sunset Blvd`。
- `prefix key` 向右结合，例如 `Apt 301`、`Suite 300`。

这意味着：

- `road suffix -> detail prefix` 不应该继续做中文式 KEY→KEY 吞噬。
- `Apt` 不该把 `Sunset Blvd` 吞成 `detail`。

5.3 英文 road 左扩必须允许跨越数字/序数字 clue

当前英文 road 的左扩在 `_left_address_floor()` 处被 structured clue 截断，导致：

- `620 8th Ave` 中 `8th` 无法被 `Ave` 吸收。
- 更靠左的 `620` 也完全丢失。

建议：

1. 为英文 `road suffix` 提供专门的左扩函数。
2. 允许其跨越与 street core 相关的 `NUMERIC/ALNUM` clue。
3. 把 `house number` 与 `street core` 作为 road 组件的前置结构，而不是简单 floor。

5.4 把 `state + ZIP` 从一个 value 中拆开

当前逗号尾下，英文 `province/state` 会因为右向扫描把 ZIP 一起吸进去，形成：

```
province: WA 98122
```

这是错误的结构化结果。

建议：

1. 英文逗号尾不应沿用中文 `VALUE` 右扫策略。
2. `city, state ZIP` 应按固定模板拆成至少两个组件：
    - `city`
    - `state`
    - `postal_code`

必要时可以把 `postal_code` 暂放进 `detail`，但不能继续并进 `province`。

5.5 州名/州缩写/国家别名要接入当前 normalize 路径

仓库中已经存在：

- `load_en_us_states()`
- `load_en_address_country_aliases()`

但当前 `normalize_pii(ADDRESS)` 没有真正使用它们。

建议：

1. 在 `normalized_pii.py` 中引入州名/州缩写统一 canonical。
2. 对 `country` 做 alias 归一，例如 `United States` / `USA` / `US`。
3. 在 `same_entity()` 中比较 canonical，而不是比较原始 value 的子串。

否则，下面这种理应成立的比较会持续失败：

- `California` vs `CA`

5.6 为英文建立独立测试矩阵

建议新增英文 detector 回归测试，至少覆盖以下结构：

1. `1425 Pine St Apt 301, Seattle, WA 98122`
2. `Apt 205, 5622 Lincoln Avenue, Seattle, WA 45283`
3. `620 8th Ave, New York, NY 10018`
4. `2900 Sunset Blvd Apt 4B, Los Angeles, CA 90026`
5. `Suite 300, 1853 North Lake Way, Detroit, MI 97385`
6. `PO Box 123, Austin, TX 78701`
7. `123 Main St, Unit B, Portland, OR 97205, United States`

测试目标不应只检查“是否有 address 候选”，还要检查：

- `address_component_trace`
- `address_component_key_trace`
- `ordered_components`
- `same_entity()`

---

6. 最终判断

如果问题是：

> 英文地址能不能直接复用中文检测方法？

我的结论是：

不能直接复用，但非常适合在中文方法的框架上做英语专用语法分支。

更具体地说：

- **复用框架**：可以。
- **复用中文链规则**：不可以。
- **复用当前元数据接口和下游归一接口**：可以。
- **必须新增英文 grammar、组件类型和测试体系**：是。

如果继续沿用现在的策略，英文地址更像是：

- “抓到一些地址片段”

而不是：

- “稳定恢复一个结构化完整英文地址”。

这也是为什么当前英文样例经常出现：

- 门牌号丢失；
- road 被 detail 吞并；
- unit 前缀被拆成独立地址；
- ZIP 被并进 state；
- 州名与州缩写无法判同一实体。

这些问题都不是词典数量不足，而是**语法方向和组件建模不对**。